Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 6

Due date: **Friday** 22 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Graph Partitioning with Julia: Load balancing for HPC [50 points]

1.1. Graph Partitioning Algorithms

1.1.1. Inertial Bisection algorithm

Listing 1 reports the implementation of the inertial bisection algorithm. This method uses the geometrical information associated with the graph vertices, namely their coordinates, to split the graph into two subdomains. The basic idea is to interpret the vertices as a set of points in the plane and to compute the direction along which the point cloud has the largest spatial spread. The graph is then partitioned by projecting the vertices onto this direction and separating them according to their position with respect to a cutting line.

Listing 1: Inertial Bisection algorithm

```
function inertial_part(A, coords)

    p = ones(Int, size(coords, 1))

    xbar = mean(coords[:, 1])
    ybar = mean(coords[:, 2])

    Sxx = sum((coords[:, 1] .- xbar) .^ 2)
    Syy = sum((coords[:, 2] .- ybar) .^ 2)
    Sxy = sum((coords[:, 1] .- xbar) .* (coords[:, 2] .- ybar))

    M = Symmetric([Syy -Sxy;
                   -Sxy Sxx])

    eigvals, eigvecs = eigs(M; nev=1, which=:SM)
    eigv = real.(eigvecs[:, 1])

    p1, p2 = partition(coords, eigv)

    p[p1] .= 1
    p[p2] .= 2

    return p
end
```

The function first initializes the partition vector `p`, assigning all vertices to the same subdomain. The number of entries of `p` is equal to the number of rows of `coords`, hence to the number of vertices of the graph.
The quantities `xbar` and `ybar` represent the coordinates of the center of mass of the vertices. The coordinates are then shifted with respect to this center of mass in order to compute the second-order moments `Sxx`, `Syy`, and `Sxy`. These quantities measure the spatial distribution of the vertices around the center of mass: `Sxx` and `Syy` quantify the spread along the coordinate directions, while `Sxy` measures the correlation between the two coordinates.

Using these moments, the matrix

$$M = \begin{pmatrix} S_{yy} & -S_{xy} \\ -S_{xy} & S_{xx} \end{pmatrix}$$

is assembled as a symmetric matrix. The eigenvector associated with the smallest eigenvalue of `M` defines the principal inertial direction of the point set. In the implementation, this eigenvector is computed by calling `eigs` with `nev=1` and `which=:SM`, which selects the eigenpair of smallest magnitude. Since the matrix is real and symmetric, the resulting eigenvector is converted to its real part and stored in `eigv`.
The actual split is then performed by the auxiliary routine `partition(coords, eigv)`. This routine projects the vertex coordinates onto the inertial direction and separates the vertices into two sets, denoted by `p1` and `p2`. These two sets correspond to the two sides of the bisecting line. Finally, the partition vector is updated by assigning label `1` to all vertices in `p1` and label `2` to all vertices in `p2`. The returned vector `p` therefore contains the bisection of the graph induced by the inertial direction.

1.1.2. Spectral Bisection Algorithm

In Listing 2, we report the implementation of the spectral bisection algorithm. Unlike inertial bisection, this method does not use the geometrical coordinates of the vertices. Instead, it exploits the connectivity structure of the graph through the graph Laplacian matrix. The main idea is to compute an eigenvector of the Laplacian, known as the Fiedler vector, and use its components to divide the vertices into two groups.

Listing 2: Spectral Bisection algorithm

```
function spectral_part(A)

    n = size(A, 1)

    if n == 1
        return ones(Int, 1)
    end

    # 1. Construct the Laplacian matrix.
    D = Diagonal(vec(sum(A, dims=2)))
    L = Symmetric(Matrix(D - A))

    # 2. Compute eigendecomposition.
    eig = eigen(L)

    # 3. Take eigenvector associated with second smallest eigenvalue.
    perm = sortperm(eig.values)
    fiedler_vec = eig.vectors[:, perm[2]]

    # 4. Balanced partition by sorting the Fiedler vector.
    order = sortperm(fiedler_vec)

    p = ones(Int, n)

    n1 = div(n, 2)
    p[order[1:n1]] .= 1
    p[order[n1+1:end]] .= 2

    return p

end
```

The function takes as input the adjacency matrix `A` of the graph and first extracts the number of vertices `n`. A special case is included for graphs with a single vertex: in this situation no meaningful bisection can be performed, and the function simply returns a partition vector containing one label.

The first step of the algorithm consists in constructing the graph Laplacian. The diagonal matrix `D` contains the degrees of the vertices, computed by summing the rows of the adjacency matrix. The Laplacian is then assembled as

$$L = D - A.$$

In the implementation, the matrix is explicitly converted to a symmetric matrix using `Symmetric(Matrix(D - A))`. This is consistent with the fact that, for an undirected graph, the Laplian is real and symmetric.
The spectral information of the graph is then obtained by computing the eigendecomposition of `L`. The eigenvalues are sorted in increasing order, and the eigenvector associated with the second smallest eigenvalue is selected. This vector is the Fiedler vector. The smallest eigenvalue of the Laplacian is associated with the constant eigenvector and does not provide a useful separation of the graph. The second smallest eigenvector, instead, contains information about the weakest connected direction of the graph and is therefore used to define the bisection.

In the theoretical version of spectral bisection, the split can be obtained by comparing the components of the Fiedler vector with a threshold, often chosen as zero or as the median value. In this implementation, a balanced partition is enforced explicitly. The entries of the Fiedler vector are sorted, and the first

`div(n, 2)` vertices in this ordering are assigned to the first subdomain, while the remaining vertices are assigned to the second subdomain. This guarantees that the two parts have approximately the same number of vertices, which is desirable in recursive graph partitioning because it avoids strongly unbalanced subdomains.

Finally, the partition vector `p` is returned. Its entries contain the labels of the two subdomains: vertices assigned label `1` belong to the first part, while vertices assigned label `2` belong to the second part.

1.1.3. Results

The script `main_bench.jl` was used to run the four bisection strategies on the toy meshes loaded by `bench_bisection.jl`. For each mesh and each partitioning strategy, we measured the bisection edge-cut, i.e. the number of graph edges whose endpoints belong to different parts. Since the partitions are constrained to be balanced, the edge-cut provides a simple measure of the quality of the interface between the two subdomains: lower values correspond to fewer inter-partition connections and therefore to less communication in a parallel graph-distributed computation.

Mesh	Coordinate	Inertial	Spectral	METIS
`3elt`	–	–	–	–
`airfoil1`	–	–	–	–
`barth4`	–	–	–	–
`crack`	–	–	–	–
`mesh1e1`	–	–	–	–
`mesh2e1`	–	–	–	–
`mesh3e1`	–	–	–	–
`netz4504_dual`	–	–	–	–
`stufe`	–	–	–	–
`ukerbe1`	–	–	–	–

Table 1: Bisection edge-cut obtained on the toy meshes loaded by `bench_bisection.jl`.

The corresponding partitions are shown in Figures 1–4 for the first four meshes, in Figures 5–9 for the additional structured meshes, and in Figure 10 for the `ukerbe1` mesh. The coordinate and inertial methods are purely geometrical: they only use the position of the vertices and therefore tend to produce almost straight cuts. This behaviour is clearly visible in the `3elt`, `barth4`, and `crack` examples. Such partitions are simple and often reasonable when the mesh is approximately uniform and the graph connectivity follows the geometry closely. However, they can become sub-optimal when the mesh has non-uniform density or when the geometry contains local features that should not be cut through.

The spectral and METIS partitions are instead based on the graph connectivity. As a consequence, they are generally more flexible: the interface is not restricted to be a straight geometrical line, but can follow weaker connections in the graph. This is particularly important for meshes such as `airfoil1`, where the local refinement around the airfoil and its wake makes a purely geometrical split less adapted to the graph structure. In this case, the coordinate and inertial cuts separate the mesh in a rather direct left-right fashion, while spectral bisection and METIS produce more irregular interfaces that better reflect the connectivity of the mesh.

Some surprising or clearly sub-optimal partitions can also be observed. In particular, for `barth4` the METIS and spectral results appear very unbalanced visually, with one part concentrated around only a small region near the centre of the mesh. A similar behaviour appears for the `3elt` mesh in the METIS and spectral plots, where one color occupies almost the entire mesh and the other is concentrated in a very small central region. This suggests that either the partition labels are not being displayed as expected, or that the returned partition is not a proper balanced bisection for these cases. Since recursive bisection relies on each split producing two meaningful subdomains, such cases should be checked carefully by verifying the number of vertices assigned to each part in addition to the edge-cut value.

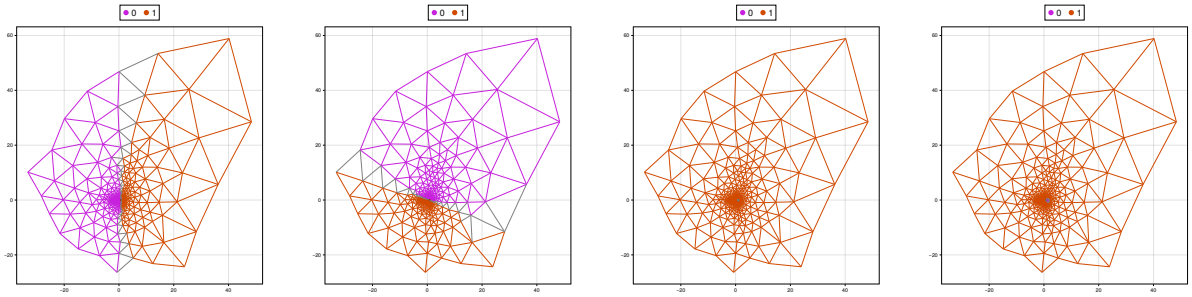
The additional structured meshes further illustrate the difference between geometrical and connectivity-based partitioning. For the regular meshes `mesh1e1`, `mesh2e1`, and `mesh3e1`, all methods produce relatively clean partitions, but the shape of the interface depends on the criterion used by the algorithm. Coordinate bisection typically produces a vertical or horizontal cut aligned with the coordinate axes, while inertial bisection may rotate the cut according to the dominant spatial direction of the mesh. Spectral bisection and

METIS may instead introduce slightly irregular interfaces when this reduces the number of cut edges.

The `netz4504_dual` and `stufe` meshes are more interesting because their geometry is less uniform. In these cases, a purely geometrical cut does not necessarily correspond to the smallest communication interface. The graph-based methods are able to adapt the separator to the local connectivity of the mesh, especially around refined or stepped regions. This is the expected behaviour: the objective of graph partitioning is not to produce a visually straight cut, but to minimize the number of edges crossing between different parts while keeping the load balanced.

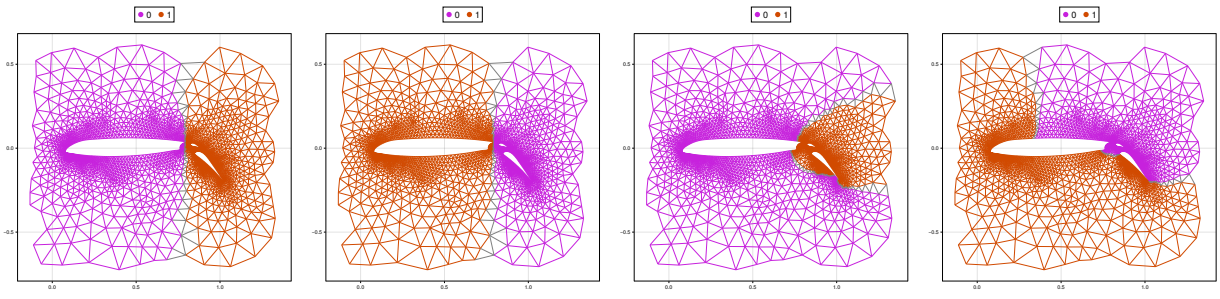
Finally, the `ukerbe1` mesh shows a geometry with a pronounced notch-like feature and several internal circular holes. In this case, the coordinate and inertial methods still produce mostly straight cuts, separating the mesh approximately into upper and lower parts. This is reasonable from the point of view of the global geometry, but it does not explicitly account for the connectivity around the notch and the locally refined region. The spectral and METIS partitions instead place the separator very close to this narrow central region. This behaviour is consistent with the graph-based objective, since cutting through a geometrical bottleneck can reduce the number of edges crossing between the two parts. However, the METIS partition appears visually suspicious, since one part is concentrated in a very small region close to the notch while the rest of the mesh belongs to the other part. As for the previous anomalous cases, this should be checked against the actual part sizes printed by the script, because a low edge-cut is meaningful only if the balance constraint is also satisfied.

Overall, the results confirm the expected trade-off between geometrical and graph-based partitioning. Coordinate and inertial bisection are cheap and produce simple cuts, but they may ignore important connectivity features. Spectral bisection and METIS can produce lower edge-cuts because they exploit the graph structure directly, but the resulting partitions may be less regular geometrically and, in some cases, should be inspected to ensure that the balance constraint has been respected.



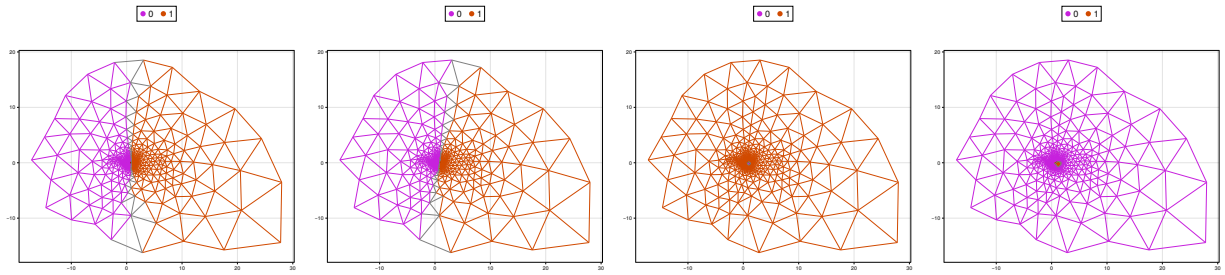
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 1: Bisection partitions obtained for the `3elt` mesh.



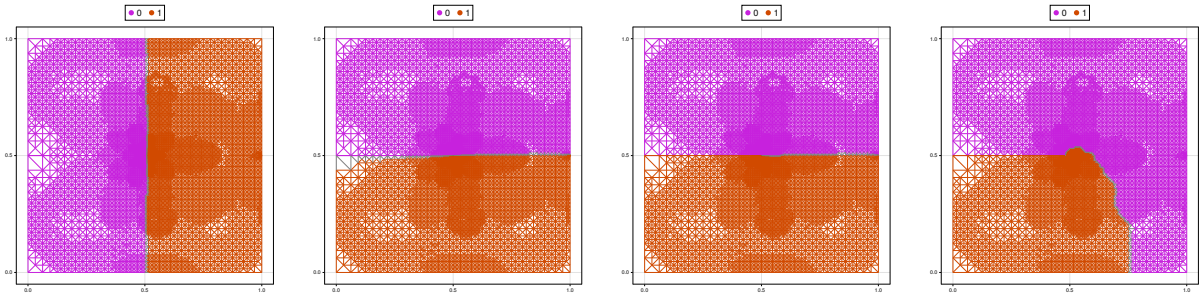
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 2: Bisection partitions obtained for the `airfoil1` mesh.



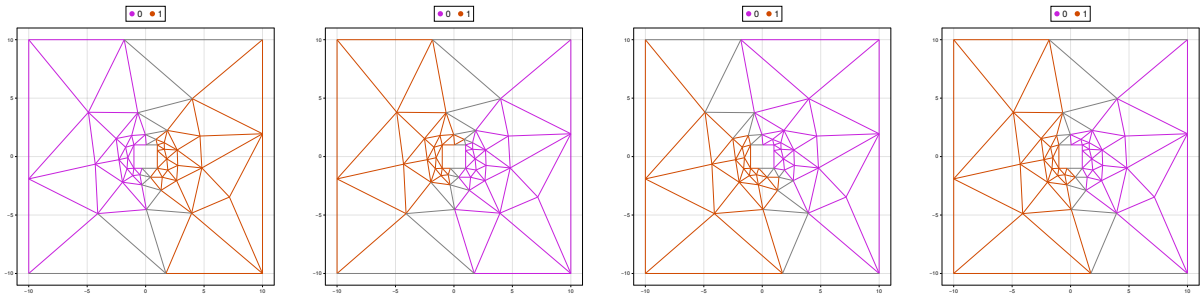
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 3: Bisection partitions obtained for the `barth4` mesh.



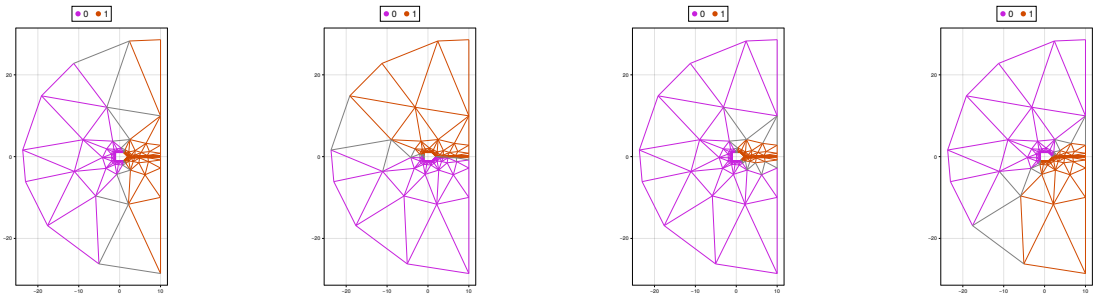
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 4: Bisection partitions obtained for the `crack` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 5: Bisection partitions obtained for the `mesh1e1` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 6: Bisection partitions obtained for the `mesh2e1` mesh.

1.2. Recursive Bisection Algorithm

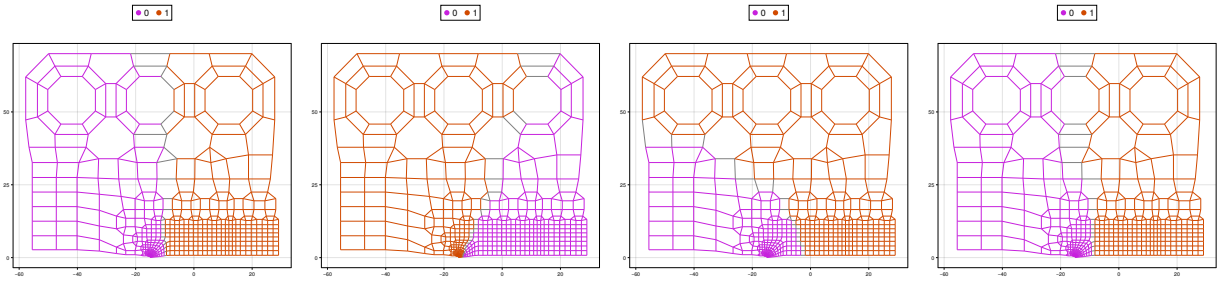
1.2.1. Implementation

Listing 3: Recursive Bisection algorithm



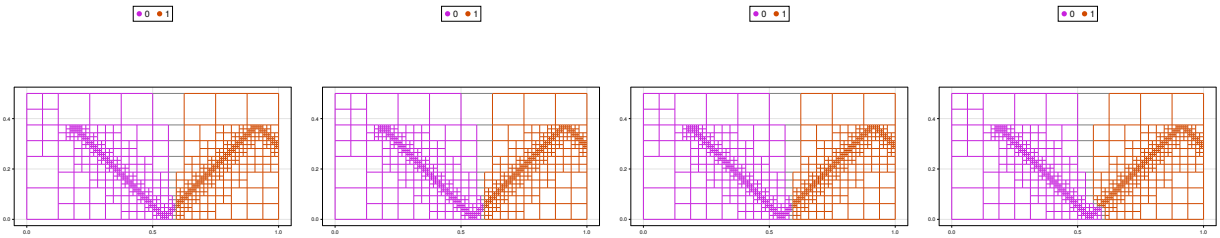
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 7: Bisection partitions obtained for the `mesh3e1` mesh.



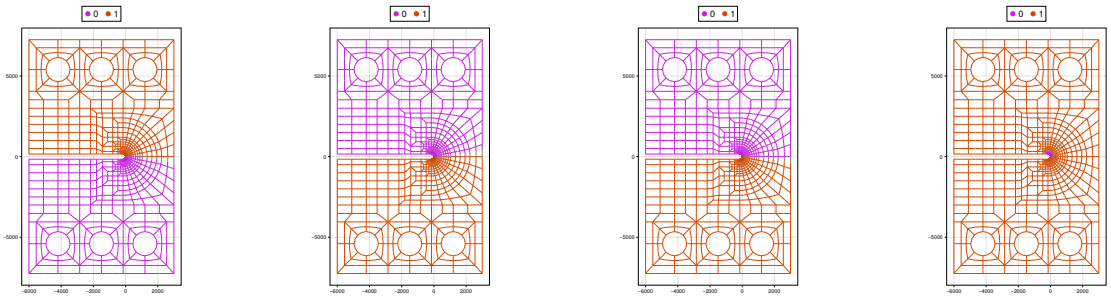
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 8: Bisection partitions obtained for the `netz4504_dual` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 9: Bisection partitions obtained for the `stufe` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 10: Bisection partitions obtained for the `ukerbe1` mesh.

```
function rec_bisection(method, levels, A, coords=zeros(0), vn=zeros(0))

    n = size(A, 1)

    # Final partition vector.
```

```
6       # p[i] is the final part assigned to vertex i.
7       p = zeros(Int, n)
8
9       function recursion(vertices, level, index)
10
11          # Stop if no more bisections are required.
12          if level == 0
13              p[vertices] .= index
14              return
15          end
16
17          # Cannot further bisect a subgraph with one vertex.
18          if length(vertices) <= 1
19              p[vertices] .= index
20              return
21          end
22
23          # Extract induced subgraph.
24          A_sub = A[vertices, vertices]
25
26          # Apply chosen bisection method.
27          if method == "spectral_part"
28              local_p = spectral_part(A_sub)
29
30          elseif method == "coordinate_part"
31              if length(coords) == 0
32                  error("Coordinates are required for coordinate bisection
                        .")
33              end
34              coords_sub = coords[vertices, :]
35              local_p = coordinate_part(A_sub, coords_sub)
36
37          elseif method == "inertial_part"
38              if length(coords) == 0
39                  error("Coordinates are required for inertial bisection.")
40              end
41              coords_sub = coords[vertices, :]
42              local_p = inertial_part(A_sub, coords_sub)
43
44          else
45              error("Unknown partitioning method: $method")
46          end
47
48          # Extract the two labels returned by the bisection method.
49          labels = sort(unique(local_p))
50
51          if length(labels) != 2
52              # This should not happen if the bisection routines are correct
                    ,
53              # but it avoids crashing on pathological small subgraphs.
54              p[vertices] .= index
55              return
56          end
57
58          vertices_1 = vertices[findall(local_p .== labels[1])]
59          vertices_2 = vertices[findall(local_p .== labels[2])]
60
61          # Safety check.
62          if isempty(vertices_1) || isempty(vertices_2)
63              p[vertices] .= index
```

```
64              return
65          end
66
67          # Each child branch contains 2^(level-1) final labels.
68          offset = 2^(level - 1)
69
70          recursion(vertices_1, level - 1, index)
71          recursion(vertices_2, level - 1, index + offset)
72
73      end
74
75      recursion(collect(1:n), levels, 1)
76
77      return p
78
79  end
```

The recursive bisection routine in Listing 3 generalizes the previous two-way partitioning methods to a $p = 2^\ell$-way partition. Instead of computing all parts at once, the graph is repeatedly split into two subgraphs. After the first bisection, each of the two parts is bisected again, and this process is repeated until the prescribed number of recursion levels has been reached. Therefore, after ℓ levels, the algorithm produces 2^ℓ final subdomains.

The function takes as input the name of the bisection method, the number of recursive levels, the adjacency matrix `A`, and, when needed, the vertex coordinates `coords`. The vector `p` stores the final partition labels. Initially all entries are set to zero. The actual recursive work is performed by the inner function `recursion(vertices, level, index)`, where `vertices` contains the indices of the current subgraph, `level` is the number of remaining bisections, and `index` is the first label assigned to the current branch of the recursion tree.

The stopping criterion is reached when `level == 0`. In this case, no further bisection is required, and all vertices of the current subgraph are assigned the current final label `index`. A second stopping condition is included for subgraphs with zero or one vertex. Such subgraphs cannot be meaningfully bisected, so they are also assigned directly to the current part. This makes the implementation robust when recursive bisection reaches very small subgraphs.

If further splitting is required, the induced subgraph is extracted as `A_sub = A[vertices, vertices]`. The chosen bisection method is then applied to this subgraph. For `spectral_part`, only the adjacency matrix is needed. For `coordinate_part` and `inertial_part`, the corresponding coordinates of the selected vertices are also extracted as `coords_sub`. The routine therefore uses the same local numbering inside the bisection method, but maps the result back to the original vertex indices through the array `vertices`.

After the local bisection has been computed, the algorithm extracts the two labels returned by the bisection method. If two valid labels are not found, or if one of the two parts is empty, the current subgraph is assigned directly to the current label. This is a safeguard against pathological cases or very small subgraphs. Otherwise, the vertices are separated into `vertices_1` and `vertices_2`, corresponding to the two children of the recursion tree.
The label offset is computed as

$$\texttt{offset} = 2^{\texttt{level}-1}.$$

This is necessary because each child branch will generate $2^{\texttt{level}-1}$ final labels. The first child therefore keeps the current starting label `index`, while the second child starts at `index + offset`. For example, with $\ell = 2$, the first bisection creates two branches: the first branch eventually receives labels 1 and 2, while the second branch receives labels 3 and 4. This indexing convention guarantees that the final partition labels are consecutive and consistent with the recursion tree.

Finally, the recursion is initialized on the full vertex set `collect(1:n)` with the prescribed number of levels and initial label 1. The returned vector `p` contains the final 2^ℓ-way partition of the graph.

1.2.2. Results

The recursive bisection procedure was tested by generating $p = 8$ and $p = 16$ partitions. For the recursive algorithms, this corresponds respectively to three and four levels of recursive bisection. In addition to the recursive versions of coordinate, inertial, spectral, and METIS bisection, we also compare against the direct k-way partitioning routine of METIS. The latter does not build the partition by repeatedly bisecting the graph, but computes the p-way partition directly.

As in the two-way case, the quality of the partition can be assessed through the total edge-cut, i.e. the number of graph edges connecting vertices assigned to different parts. For recursive partitioning, this quantity is especially important because each bisection step creates an interface, and the final edge-cut is the cumulative result of all these recursive splits.

The corresponding partitions are shown in Figures 11–**??**.

Mesh	Parts	Coordinate	Inertial	Spectral	METIS rec.	METIS k-way
`3elt`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`airfoil1`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`barth4`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`crack`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`mesh3e1`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`netz4504_dual`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`stufe`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`ukerbe1`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–

Table 2: Recursive partitioning edge-cuts for the considered meshes.

The first qualitative difference is visible when comparing the recursive geometrical methods with the graph-based methods. Coordinate and inertial recursive bisection tend to preserve the structure of the first geometrical cuts: the final subdomains are mostly obtained by repeatedly splitting the mesh along approximately straight directions. This gives partitions that are easy to interpret geometrically, but it may also create long interfaces when the mesh density is non-uniform.

For the `3elt` mesh, the recursive coordinate and inertial methods generate partitions that follow the global shape of the point cloud. However, for $p = 16$, several subdomains become very small and are concentrated near the highly refined central region. This is a consequence of applying balanced bisections recursively to a mesh with strongly non-uniform element sizes. The spectral and METIS recursive partitions are more strongly influenced by the graph connectivity. In some of the plots, especially for `3elt`, this leads to partitions that appear visually unbalanced, with many labels concentrated close to the central refinement. This should be checked carefully against the actual number of vertices per part, because a low visual area does not necessarily imply a low number of vertices in a highly refined mesh.

For the `airfoil1` mesh, the effect of the airfoil geometry and the refined wake region is more pronounced. The recursive coordinate and inertial partitions divide the domain into large geometric sectors. These partitions are regular and visually clear, but they do not necessarily follow the weakest graph connections. The spectral and METIS recursive partitions adapt more to the refined regions around the airfoil and the wake. In particular, several parts are placed around the leading edge, trailing edge, and wake, where the mesh is much denser. This behaviour is expected, since these regions contain many vertices and therefore require more subdomains if the partition is to remain balanced in terms of graph vertices rather than physical area.

For the `barth4` and `crack` meshes, the same qualitative behaviour can be observed, but the effect of the geometry is different. The `barth4` mesh has a strongly non-uniform vertex distribution, with many vertices concentrated around the central region. As a consequence, some recursive partitions appear to occupy a very small physical area near the centre of the mesh. This does not necessarily imply that the partition is unbalanced in terms of graph vertices, since the central region is much more densely discretized than the outer part of the domain. Nevertheless, these plots show a limitation of interpreting graph partitions only from their geometrical area.

The `crack` mesh is instead characterized by a localized internal feature around the centre. The recursive coordinate and inertial methods generate partitions that remain mostly aligned with successive geometrical cuts. For $p = 8$ and $p = 16$, this creates large block-like regions away from the crack and much smaller regions around the refined centre. The spectral and METIS-based methods are more sensitive to the graph connectivity near the crack and tend to isolate the refined central region more strongly. This is expected, because the high local mesh density around the crack contains many graph vertices and therefore receives several subdomains when the partition is balanced by vertex count.

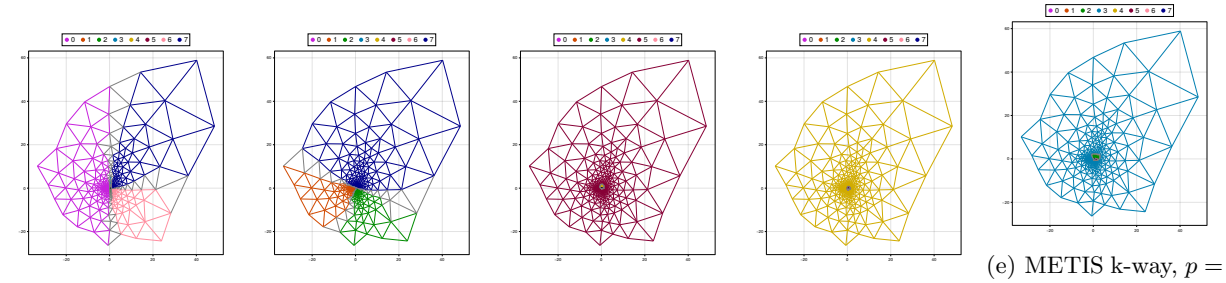
The `mesh3e1` mesh is useful as a reference case because it is much more regular than the previous unstructured examples. In this case, the recursive coordinate and inertial methods produce partitions that are mostly block-like and aligned with the geometry of the square domain. Since the mesh is close to structured, these geometrical cuts are already quite reasonable. The spectral and METIS-based methods also produce compact subdomains, but their interfaces are less constrained by coordinate directions and can adapt to the graph connectivity. For this mesh, the visual difference between the methods is less dramatic than for highly non-uniform meshes such as `airfoil1`, `barth4`, or `crack`.

The `netz4504_dual` mesh has a more complex geometry, with large coarse cells in the upper part and a much finer structured region near the bottom boundary. This makes the recursive partitioning behaviour particularly interesting. The coordinate and inertial methods tend to split the domain according to its global geometric extent, producing long vertical or horizontal regions. By contrast, the spectral and METIS partitions allocate several smaller parts close to the refined bottom region. This is consistent with the fact that graph partitioning balances the number of vertices rather than the physical area of the subdomains. Therefore, regions that are geometrically small but highly discretized may receive several partition labels.

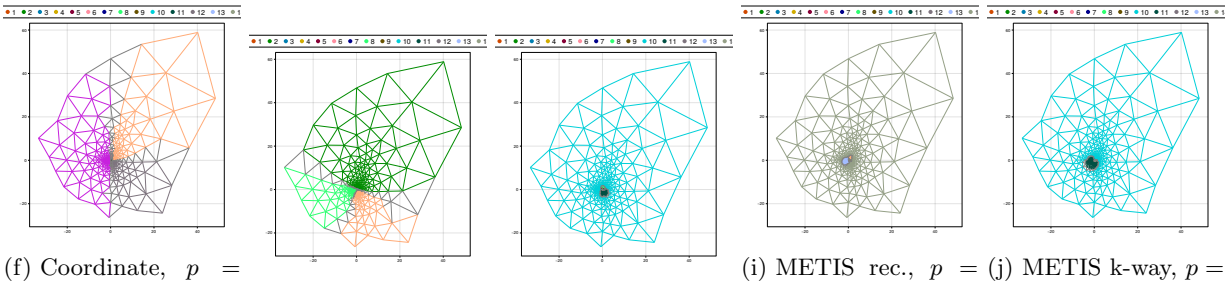
The `stufe` mesh contains a step-like geometry, with two refined diagonal regions and a relatively coarse outer rectangular structure. The recursive coordinate and inertial methods again produce partitions that are strongly driven by the global geometry. As a consequence, the parts tend to form vertical or horizontal blocks, with smaller subdomains appearing around the refined regions. The graph-based methods produce more irregular partitions, especially for $p = 16$, because the separator is allowed to follow the local mesh connectivity rather than a global coordinate direction. This gives a less visually regular decomposition, but it can reduce the number of edges crossing between parts.

The `ukerbe1` mesh is one of the most challenging examples, because it combines a large structured geometry, several circular holes, and a pronounced notch-like feature near the centre. For this mesh, the recursive coordinate and inertial methods split the domain mostly into large upper and lower or left and right regions, with additional refinement around the notch in later levels of the recursion. The spectral and METIS-based methods instead place several separators close to the central notch, which acts as a natural bottleneck of the graph. This is a reasonable behaviour for graph partitioning: cutting through a narrow, highly connected transition region can reduce the final edge-cut. However, as in the previous non-uniform meshes, some partitions appear to occupy a small physical area. This should not immediately be interpreted as poor load balance, since the local mesh density is much higher near the notch.

The comparison with METIS k-way partitioning is useful because it shows the difference between recursively applying a two-way partitioner and computing the final p-way partition directly. The k-way METIS partitions are generally more globally optimized: the algorithm can account for all p parts simultaneously, rather than committing to early bisection decisions that cannot be changed later. Therefore, one expects METIS k-way partitioning to give lower edge-cuts than recursive partitioning, especially for $p = 16$, where early suboptimal splits are amplified by the additional recursion levels.

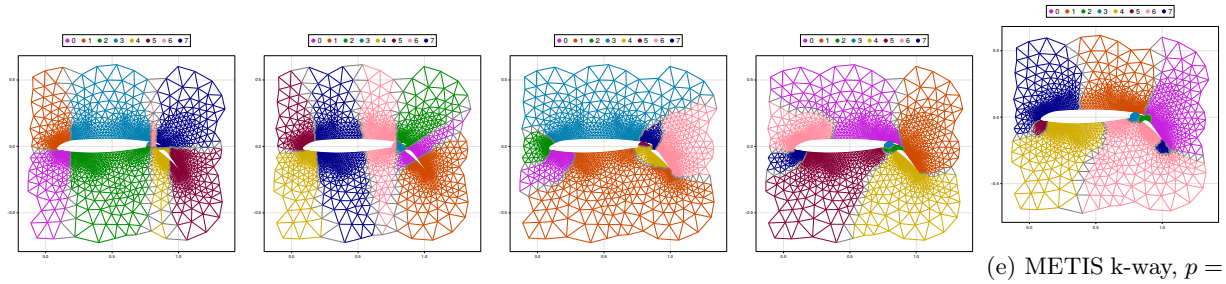


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

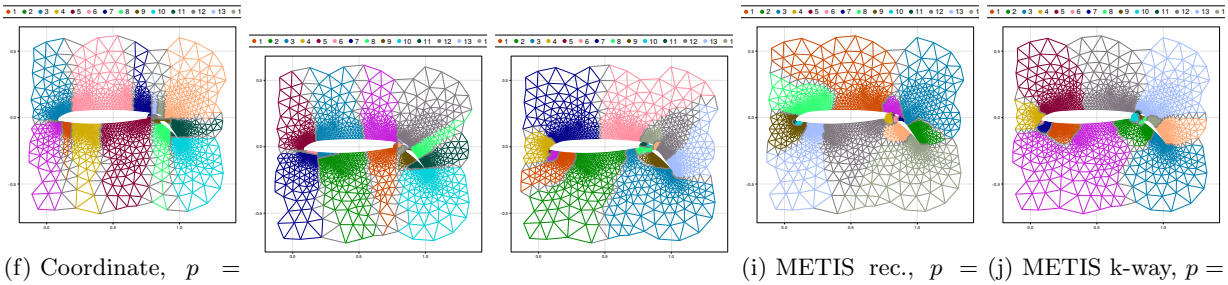


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 11: Recursive and k-way partitions obtained for the `3elt` mesh with $p = 8$ and $p = 16$ parts.

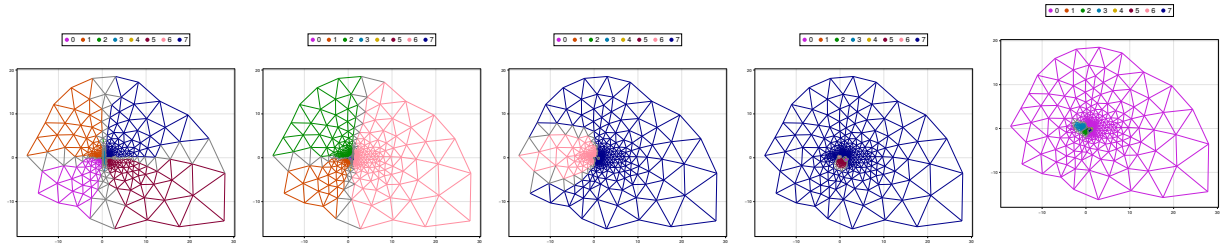


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

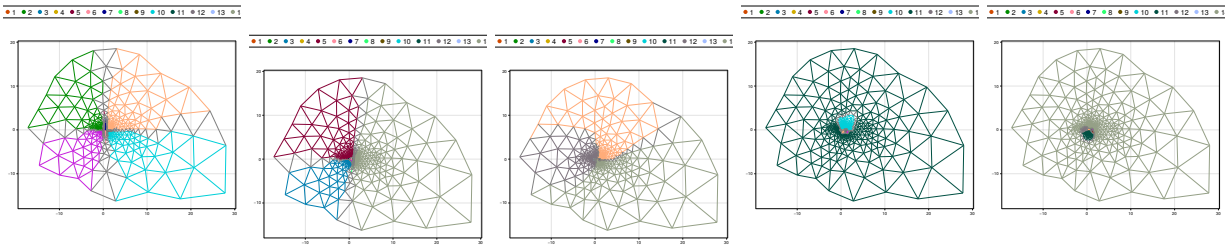


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 12: Recursive and k-way partitions obtained for the `airfoil1` mesh with $p = 8$ and $p = 16$ parts.

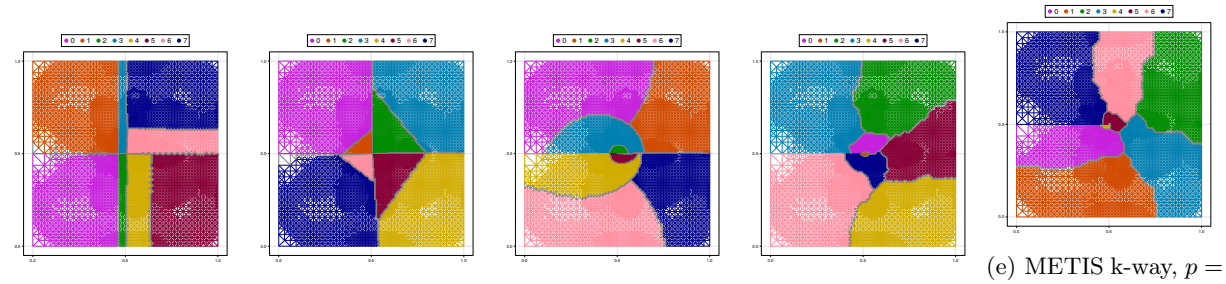


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

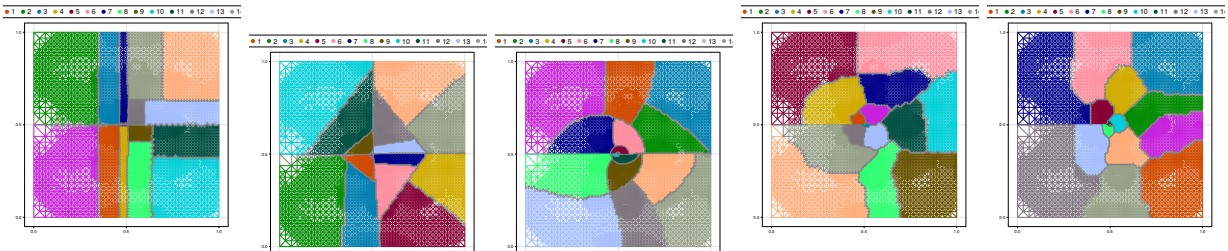


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 13: Recursive and k-way partitions obtained for the `barth4` mesh with $p = 8$ and $p = 16$ parts.

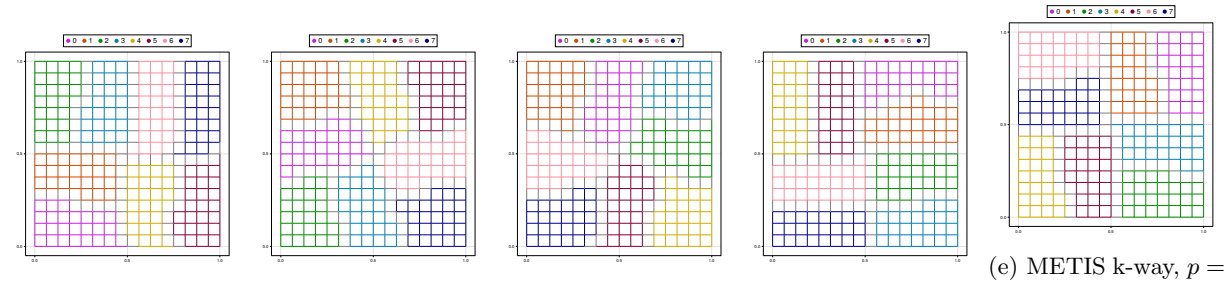


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

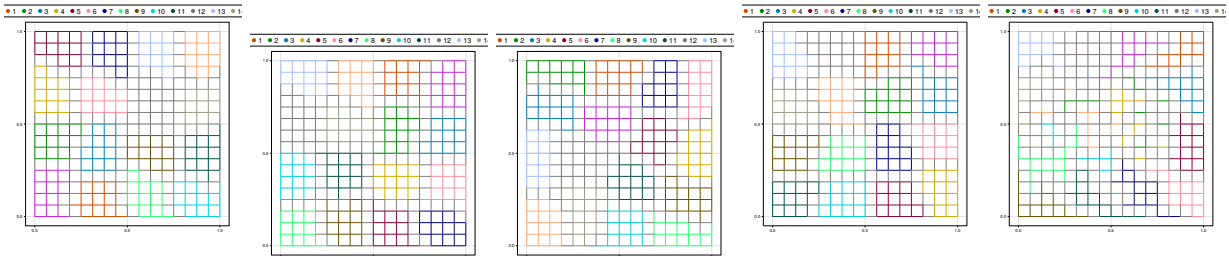


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 14: Recursive and k-way partitions obtained for the `crack` mesh with $p = 8$ and $p = 16$ parts.

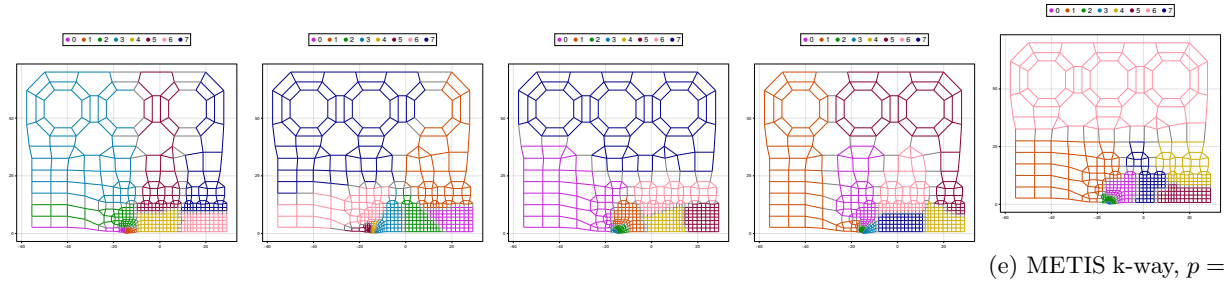


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

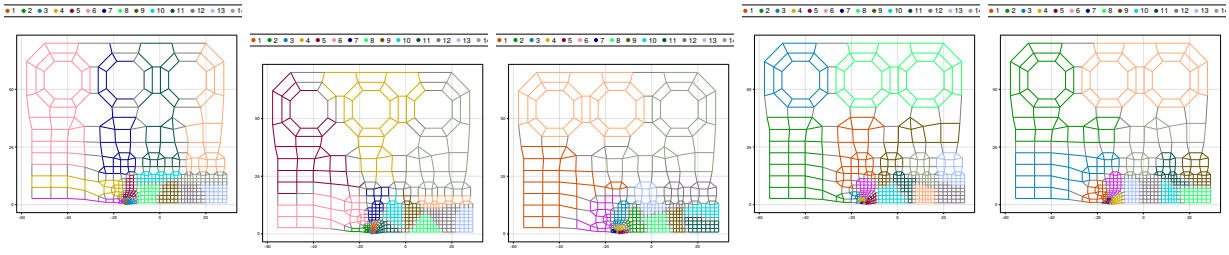


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 15: Recursive and k-way partitions obtained for the `mesh3e1` mesh with $p = 8$ and $p = 16$ parts.

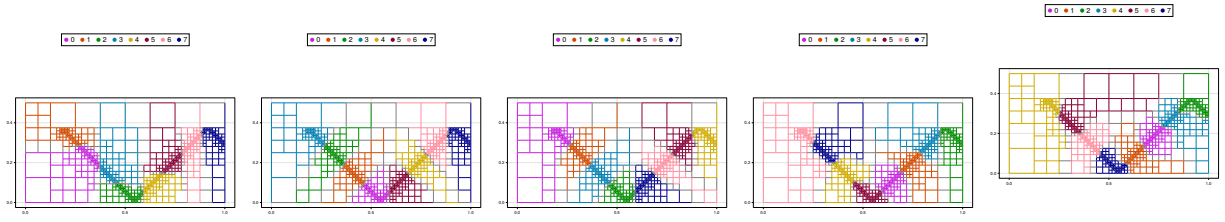


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

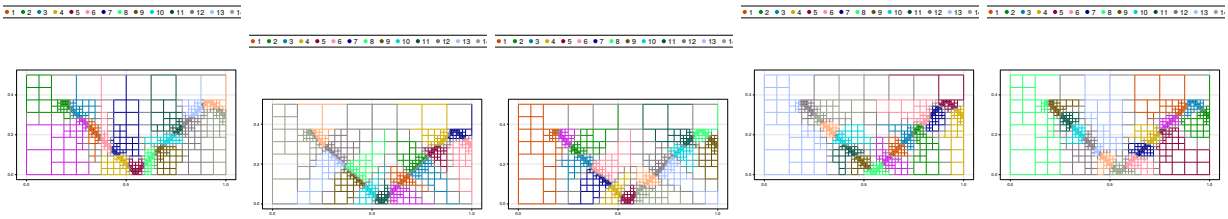


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 16: Recursive and k-way partitions obtained for the `netz4504_dual` mesh with $p = 8$ and $p = 16$ parts.

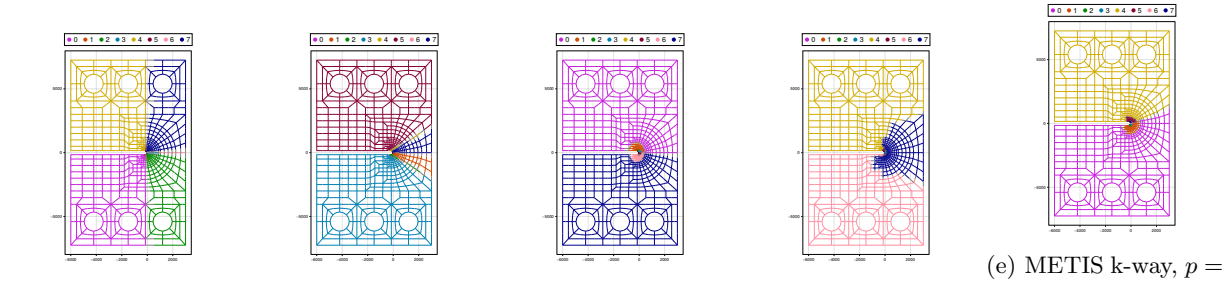


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

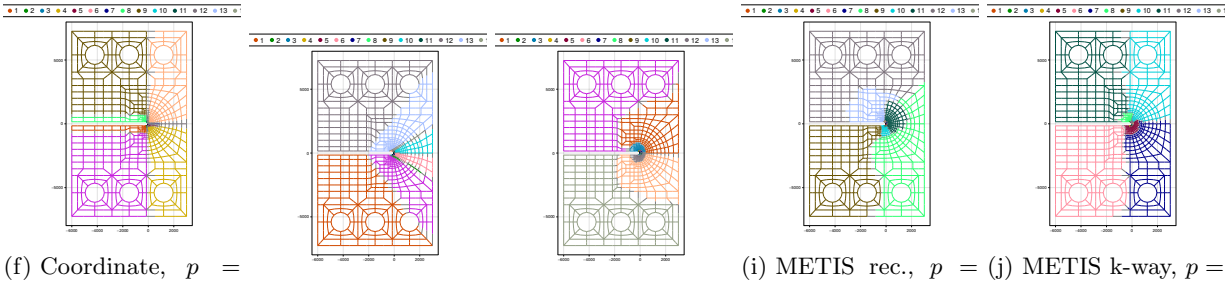


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 17: Recursive and k-way partitions obtained for the `stufe` mesh with $p = 8$ and $p = 16$ parts.



(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$



(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 18: Recursive and k-way partitions obtained for the `ukerbe1` mesh with $p = 8$ and $p = 16$ parts.

2. HPC software frameworks [50 points]

2.1. Mathematical Formulation

2.2. PETS Implementation

2.3. Visualization

2.4. Strong Scaling Analysis